Software Requirements Specification

Jackfruit Phase-1 — Student Record System

Tanmay T A, PES1UG24CS492 Uttam B S, PES1UG24CS508
Vibha Venkatesh Vasisht, PES1UG24CS526

Generated 2026-10-11

Contents

Revision History **3**

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Intended Audience and Reading Suggestions . . . 3
1.3 Product Scope . . . 4
1.4 References . . . 5

2 Overall Description **5**
2.1 Product Perspective . . . 5
2.2 Product Functions . . . 7
2.3 User Classes and Characteristics . . . 8
2.3.1 Administrator . . . 8
2.3.2 Faculty . . . 8
2.3.3 Student . . . 9
2.4 Operating Environment . . . 9
2.5 Design and Implementation Constraints . . . 10
2.6 Assumptions and Dependencies . . . 11

3 External Interface Requirements **12**
3.1 User Interfaces . . . 12
3.2 Software Interfaces . . . 14
3.3 Communications Interfaces . . . 18

4 Analysis Models **18**

5 System Features **18**
5.1 Onboarding . . . 18
5.1.1 Description and Priority . . . 18
5.1.2 Stimulus/Response Sequences . . . 19
5.1.3 Functional Requirements . . . 20
5.2 Edit Records . . . 21
5.2.1 Description and Priority . . . 21

5.2.2 Stimulus/Response Sequences . . . 22
5.2.3 Functional Requirements . . . 23
5.3 Mark Attendance (with constraints) . . . 24
5.3.1 Description and Priority . . . 24
5.3.2 Stimulus/Response Sequences . . . 24
5.3.3 Functional Requirements . . . 26
5.4 Mark Grades (with rubric) . . . 26
5.4.1 Description and Priority . . . 26
5.4.2 Stimulus/Response Sequences . . . 27
5.4.3 Functional Requirements . . . 28
5.5 Retrieval . . . 30
5.5.1 Description and Priority . . . 30
5.5.2 Stimulus/Response Sequences . . . 30
5.5.3 Functional Requirements . . . 30
5.6 Role-Based Access . . . 30
5.6.1 Description and Priority . . . 30
5.6.2 Stimulus/Response Sequences . . . 32
5.6.3 Functional Requirements . . . 33

6 Other Nonfunctional Requirements 34
6.1 Performance Requirements . . . 34
6.2 Safety Requirements . . . 34
6.3 Security Requirements . . . 35
6.4 Software Quality Attributes . . . 37
6.5 Business Rules . . . 38

7 Other Requirements 40
7.1 Database Requirements . . . 40
7.2 Legal and Regulatory Requirements . . . 41
7.3 Internationalisation and Localisation . . . 41
7.4 Reuse . . . 41
7.5 Initial Setup and Data . . . 41
7.6 Documentation and Delivery . . . 42

Appendix A: Glossary 42
A.1 Users and Roles . . . 42
A.2 Accounts and Records . . . 42
A.3 Academic Structure . . . 43
A.4 Attendance . . . 44
A.5 Grading . . . 44
A.6 Security and Technical Terms . . . 44
A.7 Project and Document Terms . . . 46
A.8 Abbreviations . . . 46

Appendix B: Field Layouts 47
Student . . . 47

Appendix C: Requirement Traceability Matrix 47

Appendix D: Stakeholder Discussion Log **47**
Persona 47
Discussion 1: Discovery Call (Monday, 14 September 2026) 47
Discussion 2: Deployment and Constraints (Thursday, 24 September 2026) 48
Discussion 3: Sign-off (Tuesday, 6 October 2026) 50

Revision History

Name	Date	Reason For Changes	Version
		Initial draft	0.1

1 Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for the **Student Record System**, release 1.0. It states what the system must do and the qualities it must have, so that the team can design, build and test it, and so that the institution it is written for can confirm it describes the system they need.

This SRS covers the whole product. All six functions are specified in Section 5: onboarding, editing records, marking attendance, marking grades with a rubric, retrieval, and role-based access. It also covers the interfaces (Section 3), the nonfunctional requirements (Section 6), and the supporting appendices. No part of the system is specified in a separate document.

1.2 Intended Audience and Reading Suggestions

This document is written for the following readers.

- **Developers** use it as the basis for design and implementation. They should read Sections 2, 3 and 5, and refer to Appendix B for field definitions.
- **Testers** derive test cases from it. They should read the functional requirements and stimulus/response sequences in Section 5, the nonfunctional requirements in Section 6, and the traceability matrix in Appendix C.
- **The course instructor and evaluators** assess the document as the Phase-1 deliverable of the Software Engineering project. They may read it in order, beginning with Section 1.3 and Appendix D.
- **The institution's stakeholder and IT team** confirm that it matches their needs and can be hosted and maintained by them. They should read Sections 1.3 and 2, the security and performance requirements in Section 6, and Appendix D.

The rest of the document is organised as follows.

Section	Contents
1. Introduction	Purpose, audience, product scope and references.

Section	Contents
2. Overall Description	The product in context: perspective, functions, user classes, operating environment, constraints, and assumptions.
3. External Interface Requirements	User interfaces, software interfaces and communications interfaces.
4. Analysis Models	Use case diagram and entity-relationship diagram.
5. System Features	The detailed functional requirements, one subsection per function (5.1 to 5.6).
6. Other Nonfunctional Requirements	Performance, safety, security, software quality attributes and business rules.
7. Other Requirements	Requirements not covered elsewhere.
Appendices	A: Glossary. B: Field Layouts. C: Requirement Traceability Matrix. D: Stakeholder Discussion Log.

Conventions used in this document.

- Each functional requirement has a unique identifier of the form `REQ-XXX-nnn` (for example `REQ-GRD-004`). The prefixes are ONB, REC, ATT, GRD, RET and RBAC, one for each function, and SEC, SAF, QUA and OTH for the security, safety, quality and other requirements in Sections 6 and 7. Business rules in Section 6.5 are numbered `BR-nnn`.
- The word "shall" marks a mandatory requirement.
- "TBD" marks information not yet available. Each TBD names the section where it will be resolved.
- Terms such as Administrator, Faculty, Anchor Faculty and Inactive have the specific meanings given in Appendix A, and are capitalised or used in those exact forms throughout.

1.3 Product Scope

The Student Record System is a web application that gives an institution one controlled place to keep its student records, attendance and grades. At present these are held in spreadsheets and departmental registers. Producing a complete picture of one student means collecting data from several offices, there is no reliable way to tell who changed a record or when, and attendance is often entered days late (Appendix D). The system replaces this with a single system in which every user sees and does only what their role permits, and every change can be traced.

Objectives. The system is intended to:

- keep one authoritative record for each student, which attendance and grades attach to;
- control access by role, so that Administrators, Faculty and Students each use only the functions and records that belong to them;
- let faculty record attendance promptly, under rules that the system enforces;
- grade each course against its own rubric, with the rubric controlled by a designated anchor faculty member and grades released to students only when they are final;

- record every change in an audit trail, so that who changed what, and when, can always be established; and
- retrieve records quickly at the scale of a whole institution, which is about 10,000 students and staff.

In scope for this release. The six functions specified in Section 5: onboarding, editing records, attendance with constraints, grading with a rubric, retrieval, and role-based access.

Out of scope for this release. Fee management, timetable generation, a parent portal and integration with the university's single sign-on service. Self-registration and email verification are also excluded. The design is intended to make later addition of these features possible without rework.

Deployment and technology. The system will be self-hosted on the institution's own servers, packaged with Docker. It is built with a React frontend, a Flask backend and a MySQL database. These choices, and the reasons for them, are recorded in Section 2.5 and Appendix D.

Relation to institutional goals. The system supports the institution's goal of replacing fragmented, department-by-department record keeping with one governed system that gives consistent reports, consistent access control and a reliable history of changes.

How the scope was set. No real customer was available for this academic project. The scope was therefore defined through three discussions with a hypothetical stakeholder, the Vice Chancellor of an institution of about 10,000 students and staff. The discussions are recorded in Appendix D, and decisions taken in them are cited where they apply. There is no separate vision and scope document.

1.4 References

1. PES University, Department of Computer Science and Engineering. *Jackfruit Mini-Project: Software Engineering Project Guidelines*, subject code UE24CS341A, 2026. This includes the SRS format (Jackfruit Phase-1) that this document follows.
2. This document, Appendix D: *Stakeholder Discussion Log* (discussions held 14 September, 24 September and 6 October 2026).
3. Government of India. *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023), which applies to the personal data held by the system.
4. React documentation. https://react.dev
5. Flask documentation. https://flask.palletsprojects.com
6. MySQL Reference Manual. https://dev.mysql.com/doc/
7. Docker documentation. https://docs.docker.com

The exact versions of the software in references 4 to 7 will be fixed at the start of implementation and recorded in the design document (see Section 3.2).

2 Overall Description

2.1 Product Perspective

The Student Record System is a new, self-contained product. It replaces the spreadsheets and departmental registers the institution currently uses for student details, attendance and grades (Appendix D). It is not a follow-on to an earlier system and is not a component of a larger one.

In this release it does not exchange data with any other institutional system. Integration with the university's single sign-on service has been deferred (Section 1.3).

The system is a three-tier web application, shown in the figure below.

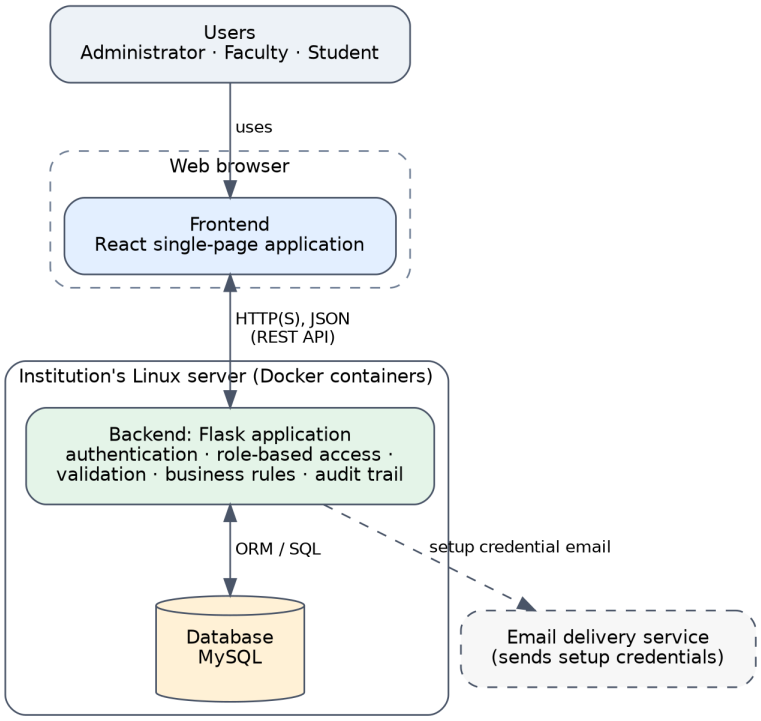


Figure 1: Three-tier architecture of the Student Record System. All components run as Docker containers on the institution's own servers (self-hosted).

Users reach the system through a standard web browser. The React frontend presents the user interface. The Flask backend is the only component that reads or writes data, and it enforces authentication, role-based access, validation and business rules on every request, so checks in the frontend are only a convenience (REQ-RBAC-003). MySQL stores all persistent data, including the audit trail, and is the same database in development, testing and deployment. The system is self-hosted because the institution's users are mostly on or near the campus, so a cloud provider's wide geographic reach offers little benefit, and the institution's own IT team can maintain it (Appendix D).

Interfaces to the outside. The system has only two:

- the users' web browsers (Sections 3.1 and 3.3); and
- an email delivery service, which sends a new user the setup credential that lets them set their own password. The provider is TBD in Section 6.3.

How the major parts relate. Account management and role-based access (5.1 and 5.6) underlie every other function, since nothing can be done without an authenticated account that has the right role. Student records (5.2) are the foundation that attendance (5.3) and grading (5.4) attach to. Retrieval (5.5) reads all of them. The audit trail records changes made through every function.

Because deferred features such as fee management and a parent portal are expected to follow, the parts above are kept separate so that new functions can be added without changing the existing ones.

2.2 Product Functions

The system provides the functions below. Each is specified in detail in the Section 5 subsection shown.

Function	Summary	Main users	Section
Onboarding	Administrators create Student and Faculty accounts. Each new user sets their own password through a secure one-time setup and then logs in with their email address. There is no self-registration.	Administrator	5.1
Edit Records	Administrators correct and update student records. Every edit is audited. A student who leaves is set to Inactive, which blocks login but keeps the record, attendance and grades.	Administrator	5.2
Mark Attendance	Faculty mark attendance for the class sessions of courses they teach. The system enforces the attendance rules: attendance can be marked only for scheduled sessions, a student cannot be present in two overlapping sessions, and entries can be corrected only within a limited window, after which only an Administrator can amend them.	Faculty, Administrator	5.3
Mark Grades	The anchor faculty of each course defines that course's rubric. Faculty teaching the course enter marks against it, and the system calculates each student's weighted total. The anchor faculty publishes the grades, after which students can view their own. Later corrections need a recorded reason.	Faculty, Student	5.4
Retrieval	Users search for and view student records, attendance and grades, limited to what their role allows.	Administrator, Faculty, Student	5.5
Role-Based Access	Every account has exactly one role: Administrator, Faculty or Student. Anything not explicitly permitted to a role is denied, and the backend enforces this on every request. Administrators can change roles between Student and Faculty and can deactivate or reactivate accounts.	All users	5.6

Capabilities that apply across the functions.

- **Authentication.** Users log in with their registered email address and password. See Sections 5.1 and 6.3.
- **Audit trail.** Changes to records, rubrics, marks, attendance and roles, and denied requests, are recorded and cannot be altered through the application. See Sections 5.2, 5.4, 5.6 and 6.3.

- **Performance and security.** Retrieval and updates must meet the response-time target, and student data must be protected in line with the DPDP Act, 2023. See Sections 6.1 and 6.3.

2.3 User Classes and Characteristics

The Student Record System has three user classes: Administrator, Faculty and Student. Every account holds exactly one of these roles, and access to functions and records depends on that role (Section 5.6). Accounts are created only by an Administrator (Section 5.1), so a person cannot register themselves into any user class. Administrator accounts are pre-seeded during initial system setup and cannot be created or assigned through the application. The detailed permissions of each class are specified in Section 5.6 and are not repeated here.

The descriptions of technical proficiency and frequency of use below are assumptions made for this project, not established requirements. They are based on an institution of about 10,000 students and staff (Appendix D). The user classes, their access levels and their limitations follow from Sections 5.1 and 5.6.

2.3.1 Administrator

- **Purpose:** Administers the system. The Administrator creates and maintains user accounts and keeps the student data accurate.
- **Main functions:** Creates, updates, deactivates and reactivates Student and Faculty accounts; changes roles between Student and Faculty; edits student records; views all attendance and grades and amends attendance as allowed by Section 5.3; designates and reassigns the anchor faculty of each course.
- **Access level:** Highest privilege, with deliberate limits (see below). The Administrator can access all relevant student and faculty records.
- **Information accessed:** Account details of all users, and the records, attendance and grades (including unpublished grades) of all students.
- **Technical proficiency (assumed):** Moderate. The Administrator is expected to be comfortable with administrative web applications and data entry, but needs no programming knowledge.
- **Frequency of use (assumed):** Periodic. Use is concentrated around account setup and record maintenance, with ad hoc use whenever accounts or records need correction.
- **Important characteristics and limitations:** Only an Administrator can manage accounts and roles. An Administrator cannot create, alter or delete rubrics, and cannot enter or amend marks, which keeps grading with the faculty who teach the course. Administrator accounts exist only because they were pre-seeded at initial setup. The Administrator never sets or knows a user's password, because each user sets their own.

2.3.2 Faculty

- **Purpose:** Records and maintains the academic information of the students in the courses they teach.
- **Main functions:** Views the students enrolled in the courses they teach, marks and edits attendance (within the rules in Section 5.3), enters and edits marks against the course rubric, and views the relevant academic records of those students.

- **Anchor faculty:** For each course, an Administrator designates one of the faculty teaching it as the anchor faculty. In addition to the functions above, the anchor faculty creates and alters that course's rubric, publishes its grades, and amends published marks (Section 5.4). Other faculty teaching the course can view the rubric but not change it.
- **Access level:** Restricted to the courses they teach and the students enrolled in them.
- **Information accessed:** Attendance, marks and academic records of the students enrolled in their courses only.
- **Technical proficiency (assumed):** Basic to moderate. Faculty are expected to use standard web forms and tables for data entry and lookup.
- **Frequency of use (assumed):** Regular during the academic term, mainly for attendance and mark entry. Use peaks at the start and end of class periods, when attendance is marked.
- **Important characteristics and limitations:** A Faculty member cannot access students outside their courses, cannot manage accounts or roles, and cannot edit student records. Only the anchor faculty of a course can change its rubric.

2.3.3 Student

- **Purpose:** Views their own academic information held by the institution.
- **Main functions:** Views their own profile, attendance, published grades and academic records.
- **Access level:** Lowest privilege. Access is limited to the Student's own records and is read-only.
- **Information accessed:** Their own profile, attendance, published grades and academic records only. Grades are visible only after the anchor faculty has published them.
- **Technical proficiency (assumed):** Basic. The Student is expected to be able to log in and read information presented on screen.
- **Frequency of use (assumed):** Occasional, typically to check attendance or grades. Use rises sharply when results are published.
- **Important characteristics and limitations:** A Student cannot access any other student's records and cannot modify their own grades or attendance. Students are by far the most numerous user class, and their access depends on an Administrator creating their account. A Student whose account is Inactive cannot log in.

Priority of user classes. Faculty are the most important class to satisfy. They enter most of the data, so the accuracy and timeliness of attendance and grades depend on the system being quick and easy for them to use. Administrators come next, because correct accounts and records underlie every other function. Students are the largest class but only read information, so their needs centre on simple login and clear results.

2.4 Operating Environment

Server environment. The system runs on servers owned and operated by the institution (decision of 24 September 2026, Appendix D). It is packaged as Docker containers, one each for the frontend, the backend and the MySQL database, defined together in a Docker Compose file. The host therefore needs only to support Docker. The host is assumed to be a Linux server. The system must coexist with the other systems the IT team already runs, and must not require changes to them.

Capacity. The system is designed for about 10,000 users, with a few hundred concurrent users at

peak times, such as when faculty mark attendance at the start and end of class periods and when results are published (Appendix D). Hardware sizing is TBD and will be given in the deployment documentation.

Client environment. Users need only a standard web browser, namely a current version of Chrome, Firefox or Edge. No client software is installed. The interface is designed mainly for desktop and laptop screens. Whether phones and tablets are supported is TBD in Section 3.1.

Network. Users reach the system over the institution's network. Nearly all users are on or near the campus (Appendix D), so the system does not need to serve a geographically distributed user base or balance load across regions. Whether it is also reachable from off-campus is a matter for the IT team's network arrangements and is not specified here. Browsers communicate with the server over HTTP(S) (Section 3.3).

Other software. Apart from the software inside its own containers, the system relies on the institution's backup facilities, since the database is backed up regularly (Appendix D). The system also sends the setup credential by email, so the server must be able to reach an email delivery service (provider TBD in Section 6.3).

2.5 Design and Implementation Constraints

The following constraints limit the options available to the developers. Where a constraint comes from the stakeholder discussions, the date is given (Appendix D).

Deployment

- The system shall be self-hosted on the institution's own servers and packaged with Docker (24 September 2026). Cloud and serverless deployment were considered and rejected, so the design must not depend on any cloud provider's services.
- The software shall be open source and maintainable by the institution's existing IT team, with no paid licences.

Technology

- The frontend shall be built with React and the backend with Flask (Python).
- The database shall be MySQL in every environment: development, testing and deployment. No other database is used.
- The frontend and backend shall communicate through a REST API using JSON (Section 3.2).
- Exact versions of all components shall be pinned in the project's dependency files and recorded in the design document.

Architecture

- The backend shall be the only component that accesses the database. The frontend shall never access it directly.
- All authentication, authorisation and validation shall be enforced in the backend. Checks in the frontend are for convenience only (REQ-RBAC-003).
- Operations that change several rows together shall run in a single database transaction, and editable records shall carry a version number so that stale updates are detected (Section 3.2).
- The design shall be modular and extensible, so that the deferred features (fee management, timetable generation, a parent portal and single sign-on integration) can be added later with-

out reworking the existing functions (6 October 2026).

Security and privacy

- Passwords shall be stored only as salted, adaptive, one-way hashes, never in plaintext (REQ-ONB-016). The algorithm is defined in Section 6.3.
- Access shall be role-based and deny by default (Section 5.6).
- Student records are personal data and shall be handled in compliance with the Digital Personal Data Protection Act, 2023 (24 September 2026).
- The audit trail shall be append-only and shall not be modifiable through the application.
- Student records shall not be permanently deleted through the application (REQ-REC-009). Their retention period follows the university's records policy (TBD).
- The database shall be backed up regularly.

Performance

- Record retrieval shall take under two seconds under normal load, with the system designed for about 10,000 users and a few hundred concurrent users at peak (24 September 2026). See Section 6.1.

Development process (course guidelines)

- Development shall follow an Agile process, using Git and GitHub with Jira or a similar tool for user stories and the backlog.
- A CI/CD pipeline (Jenkins), code reviews and security validation are mandatory.
- Automated tests shall use pytest, with unit test reports showing code and branch coverage, and static analysis (SonarQube or similar) shall be applied.
- Documentation is as important as code, and the design shall be modular, maintainable and debuggable.
- Python code shall follow PEP 8 and JavaScript code a standard ESLint configuration, checked automatically in the CI pipeline. The course guidelines do not prescribe a coding standard.

2.6 Assumptions and Dependencies

Each assumption below is also noted where it matters. If one proves wrong, the requirements that depend on it must be revisited.

Assumptions

- **No real customer.** This is an academic project, and no real customer was available. A hypothetical stakeholder, the Vice Chancellor of an institution of about 10,000 students and staff, stands in for the customer, and the scope and design decisions were fixed in three discussions recorded in Appendix D. This is itself an assumption, not a footnote.
- **One institution, many terms.** The system serves a single institution, and its data accumulates across terms.
- **Users are mostly on or near campus,** on the institution's network, as stated in Section 2.4.
- **The IT team will host and maintain the system.** It is assumed to have the skills to run Docker containers and MySQL, to provide the servers and network, and to take regular backups (confirmed in the discussions of 24 September and 6 October 2026).
- **Courses, terms, teaching assignments and enrolments exist.** Grading and role-based access depend on them. How they are created is TBD (Sections 5.4 and 5.6).

- **A schedule of class sessions is available.** Attendance can be marked only for scheduled sessions, and timetable generation is out of scope. How the schedule reaches the system is TBD (Section 5.3).
- **Every user has a unique email address** and can receive email at it, since the setup credential is sent by email. Email verification is out of scope, and the email provider is TBD (Section 6.3).
- **The institution will supply its grade bands and its records retention period,** which are TBD (Sections 5.4 and 2.5).
- **At least one Administrator account stays Active.** Administrator accounts cannot be created through the application, so the institution must keep one usable (see the TBD in Section 5.6).
- **Users' browsers are current, and their technical proficiency is as described in Section 2.3.**

Dependencies

- **Open-source components:** React, Flask, MySQL, Docker, a password-hashing library (bcrypt) and an ORM (SQLAlchemy, to be confirmed in the design document). They must remain available and continue to receive security fixes. Their versions are pinned (Section 3.2).
- **The institution's infrastructure:** its servers, network and backup facilities.
- **An email delivery service** for sending setup credentials, whose provider is TBD in Section 6.3.
- **The DPDP Act, 2023.** The system's obligations may change if the law or its rules change.
- **The institution's policies** on grading and on records retention.

Apart from the email delivery service, the system depends on no other institutional system or external service in this release.

3 External Interface Requirements

3.1 User Interfaces

The Student Record System is accessed by Admin, Teacher and Student users through a web-based frontend (Sections 2.3 and 5.6). Wireframes and mockups have not been produced for this version. Screen layouts and the interfaces for Sections 5.2 to 5.5 (records, attendance, grades, retrieval) are TBD until those sections are finalised.

Interfaces required by the current requirements

Interface	Used by	Logical characteristics	Reference
Login	All roles	Takes the registered email address and password. Authentication failures show one generic message that does not reveal which part was wrong. A user with a missing, expired or invalid session is directed here.	REQ-ONB-018, REQ-SEC-002, REQ-RBAC-006
Initial password setup	New Student/Teacher	Takes a password and a confirmation, using a single-use setup credential. On success the user is directed to log in separately and no session is created.	REQ-ONB-012 to 017
Account creation	Admin	Takes role, unique identifier (SRN or Staff ID), name, email and program/department details (fields in Appendix B). No password field is shown. A confirmation shows identifier, name and role only.	REQ-ONB-001 to 012, REQ-ONB-019
Account and role management	Admin	Provides update and deactivation of accounts and role assignment. Layout is TBD.	REQ-RBAC-007, 016 to 018
Assigned-student view	Teacher	Lists the students assigned to the Teacher. Detailed screens for attendance and grade entry are TBD (Sections 5.3 and 5.4).	REQ-RBAC-009

Interface	Used by	Logical characteristics	Reference
Own-record view	Student	Shows the Student's own profile, attendance, grades and academic records, read-only.	REQ-RBAC-012, 013

Role-specific behaviour

- The frontend displays only the functions permitted to the logged-in user's role (REQ-RBAC-004). The frontend is a convenience layer, and the backend enforces authorization on every request (REQ-RBAC-003).
- A request the user's role does not permit, including access to another user's record, produces an "Access Denied/Unauthorized" message and changes no data. Nonexistent and unauthorized records get the same response (REQ-RBAC-014).

Message conventions

- **Validation errors:** The message identifies every missing or invalid field and keeps the entered non-password values (REQ-ONB-008).
- **Duplicate identifier or email:** The message names the duplicated field and discloses no other data of the existing account (REQ-ONB-010).
- **Password rules and setup credential:** The message states which password rule failed, or that the credential is invalid (REQ-ONB-014, REQ-ONB-015).
- **Unexpected or authorization-check errors:** Only a generic message is shown, without stack traces, file paths, query text or other internal details (REQ-SEC-016, REQ-RBAC-015).
- **Passwords and hashes:** These are never displayed on any screen (REQ-SEC-004).

3.2 Software Interfaces

The Student Record System is a three-tier web application: a React frontend, a Flask backend and a MySQL database, packaged as Docker containers on a Linux host (Sections 2.4 and 2.5). This section describes the software components the product connects to and the data that crosses each connection. Protocol-level details (HTTP conventions, encryption, message formats) are covered in Section 3.3, and detailed API definitions will be given in the design document. Requirement identifiers in brackets point to the requirements that govern each interface: `REQ-SEC` identifiers are in Section 6.3, and the others are in Section 5.

Software components and interfaces

Component	Data exchanged	Purpose and conventions
Frontend Backend	Requests carrying user input, and responses carrying displayed records, confirmations and error messages, as JSON over HTTP(S). With each request the frontend sends the session identifier and, on state-changing requests, an anti-forgery token.	The React frontend presents role-appropriate functions and runs in a current version of Chrome, Firefox or Edge. The Flask backend exposes a REST API, validates all input and authorises every protected request through one common enforcement layer (REQ-SEC-011, REQ-SEC-013). The client cannot supply its own role, identity or ownership claim (REQ-RBAC-005).
Backend Database	Data categories established by this SRS: user accounts (identifier, name, email, role, account status), password hashes, setup credentials, student records, courses, course offerings, teaching assignments, enrolments, class sessions, attendance, rubrics, marks, and audit-trail entries.	MySQL stores all persistent data, in every environment (development, testing and deployment). The backend accesses it through an ORM such as SQLAlchemy (to be confirmed in the design document). User input reaches the database only as bound parameters (REQ-SEC-014). Account creation is atomic (REQ-ONB-020), and the uniqueness of SRN, Staff ID, Administrator ID and email is guaranteed even for simultaneous requests (REQ-ONB-009). The database schema is defined in the design document.
Backend Session store	Server-side authenticated session state and account status.	The server keeps the session state and gives the client only an opaque session identifier that contains no role or personal data. Role and account status are read from stored data on every protected request (REQ-SEC-008). The storage mechanism is TBD.
Backend Password-hashing library	Plain password in, salted hash out.	The backend hashes passwords with bcrypt, a unique salt and a cost factor of at least 10 (REQ-SEC-003, REQ-ONB-016).
Backend Email delivery service	Outbound email carrying the initial-password setup credential to the user's registered address.	Used only for onboarding (Section 5.1). No student records are sent by email (REQ-SEC-006, REQ-SEC-018). The email provider is TBD.
Docker and Docker Compose	Container images and configuration.	Packages the frontend, backend and MySQL database as containers on the institution's Linux server. A Compose file defines the services and the network between them.
Continuous integration (Jenkins) and pytest	Source code in; test results and static-analysis findings out.	The pipeline runs the pytest suite against the backend, with MySQL running in a container, and runs static code analysis. A release is blocked while high-severity findings remain (REQ-SEC-019). This is not a runtime interface.

Exact versions of all components will be pinned in the project's dependency files (`requirements.txt`, `package.json`) and recorded in the design document. The current stable release of each component at the start of implementation will be used.

No third-party APIs are used, and student and user data is not disclosed to third parties (REQ-SEC-018).

Data items exchanged between the frontend and backend

Data item	Direction	Purpose
Login credentials	In	Authenticate a user. The backend creates an authenticated session and returns an opaque session identifier, together with what the frontend needs to show the functions allowed for the user's role (Section 6.3).
Session identifier and anti-forgery token	In	Sent with each protected request to identify the authenticated session. The anti-forgery token is sent on state-changing requests.
Setup credential and new password	In	Submitted by a new user to set their own password (Section 5.1).
Student record fields	In and out	Create a record (Section 5.1), submit edits (Section 5.2), and display a record. Field definitions are in Appendix B.
Record version number	In and out	Sent with every edit so that the backend can detect stale updates (REQ-REC-007, REQ-GRD-015).
Account status and role changes	In	Submitted by an Administrator to deactivate or reactivate an account, or to change a role between Student and Faculty (Section 5.6).
Rubric definition (component name, maximum mark, weight)	In and out	Submitted by the anchor faculty to define or change a rubric. Returned to display the rubric (Section 5.4).
Anchor faculty assignment (course, faculty member)	In and out	Submitted by an Administrator to designate or reassign the anchor faculty for a course.
Marks (student, component, mark)	In	Submitted by faculty when entering or amending marks.
Grade results (marks, maximums, weights, weighted total, published and amended status)	Out	Returned to faculty and Administrators for the gradebook, and to a student for their own published results.
Amendment reason	In	Required text sent when a published mark is amended (REQ-GRD-021).
Attendance entries	In and out	Marking and viewing attendance (Section 5.3).
Search criteria and result sets	In and out	Retrieval of records, attendance and grades (Section 5.5).

Data item	Direction	Purpose
Audit-trail entries	Out	Returned to the users permitted to review the history of changes (who may do so is defined in Section 6.3). They are written only by the backend and never submitted by the frontend.
Error messages	Out	Returned with the reason a request failed, such as invalid input, access denied or a conflicting update. They never include data the user is not permitted to see.

Data shared across components

All persistent data is held in the database and shared between features through the following entities. Field-level layouts are in Appendix B, and the entity-relationship diagram is in Section 4.

- **User and Role:** the login identity (email), the role (Administrator, Faculty or Student), the account status (Active or Inactive) and the password hash. Used by every feature.
- **Setup Credential:** the single-use, time-limited token that lets a new user set their password (Section 5.1).
- **Student:** the student record created in Section 5.1 and edited in Section 5.2. Referenced by enrolment, attendance and grades.
- **Course and Course Offering:** a course in a particular term. Each offering has one anchor faculty and one or more teaching faculty.
- **Teaching Assignment:** links a Faculty member to a course offering they teach. It decides which students a faculty member can view and which courses they can mark attendance and marks for.
- **Enrolment:** links a student to a course offering.
- **Class Session:** a scheduled meeting of a course offering (Section 5.3).
- **Rubric and Rubric Component:** belongs to a course offering. Components hold the name, maximum mark and weight.
- **Mark:** one mark per student per rubric component, with its publication and amendment status.
- **Attendance Record:** one record per student per class session (Section 5.3).
- **Audit Trail:** an append-only record of every change to student records, rubrics, marks, attendance and roles, and of denied requests and failed authentications.
- **Authenticated Session state:** held in the session store, and not in the database unless the design document decides so. **Implementation constraints on data sharing**
- The frontend shall never access the database directly. All data access goes through the backend API, which re-checks the user's role and ownership on every request, regardless of what the user interface displays.

- Operations that change several rows together, such as saving an edited record with its audit entry, or changing a rubric and recalculating totals, shall run in a single database transaction so that they succeed or fail as a whole.
- Every editable record carries a version number so that concurrent edits can be detected.

3.3 Communications Interfaces

- **Client–server communication:** Users reach the system through a web browser. All client–server communication uses an encrypted connection (HTTPS). Unencrypted requests are rejected or redirected, and the session identifier is sent only over the encrypted connection and in a form not readable by client-side scripts (REQ-SEC-010). Protocol version and certificate arrangements are TBD.
- **Request/response model:** The frontend sends each user action to the backend and shows the backend's response. Every protected request carries the session identifier and is checked for authentication and authorization before processing (REQ-RBAC-003, REQ-RBAC-006). State-changing requests also carry a valid anti-forgery token tied to the session (REQ-SEC-015). A REST/JSON message format and API route conventions have not been established and are TBD.
- **Session communication:** The session identifier is opaque, is reissued at every successful login, and expires after 30 minutes of inactivity or 8 hours after login. A logged-out or expired session is rejected (REQ-SEC-008, REQ-SEC-009).
- **Backend–database communication:** The backend communicates with the database as described in Section 3.2. The connection mechanism and its security are TBD.
- **Email communication:** Email is used only to deliver the single-use initial-password setup credential, which is valid for 24 hours, to the user's registered address (REQ-SEC-006). Email contains no student records (REQ-SEC-018). Email verification, password-reset email and other notifications are out of scope in this version (Sections 5.1 and 6.3).
- **Other interfaces:** No communication with external systems or third-party services is required in this version. Transfer rates and synchronization requirements are not specified.

4 Analysis Models

TODO

5 System Features

5.1 Onboarding

5.1.1 Description and Priority

Description. Onboarding is how user accounts are created in the Student Record System. An authenticated Administrator creates Student and Faculty accounts and assigns each account a role. Self-registration is out of scope. The Administrator never sets or knows the user's password. The user sets their own password through a secure initial-password setup, then logs in separately using their registered email address and password.

Priority: High. Every other feature needs an existing, authenticated account.

Preconditions. - A pre-seeded Administrator account exists (created during initial system setup). - The Administrator performing onboarding is authenticated. **Inputs.**

Account type	Unique identifier	Other required fields
Student	SRN	Name, email, academic/programme details
Faculty	Staff ID	Name, email, department details
Administrator (pre-seeded only)	Administrator ID	Name, email, password

The Administrator also selects the role (Student or Faculty). Field lengths, data types and the final field list are defined in Appendix B.

Assumptions and dependencies. - What each role may do after onboarding is defined in Section 5.6. - The hashing algorithm, session handling, setup-credential validity period and delivery mechanism are defined in Section 6.3 (TBD). - Email verification is out of scope. - Identifier and email comparisons ignore letter case and leading/trailing whitespace.

5.1.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Administrator submits a valid account-creation request with a unique identifier and email.	Administrator authenticated; required fields valid; identifier and email unique.	Account created as Active; password setup initiated; Administrator sees a confirmation. No session is created for the new user.
SR-2	Administrator submits a request with a missing or invalid field.	Field validation fails.	No account created; every failing field is identified; entered non-password values are retained.
SR-3	Administrator submits an SRN, Staff ID, Administrator ID or email already in use.	Uniqueness check finds a match.	Request rejected; existing account unchanged; error names the duplicated field.
SR-4	A non-Administrator or unauthenticated user attempts account creation.	Authentication/role check fails.	Access denied; no account created.

#	Stimulus (trigger)	System validation	System response
SR-5	New user submits a valid password and matching confirmation using a valid setup credential.	Credential valid, unused, unexpired; password rules met.	Password hashed and stored; credential invalidated; user directed to log in separately.
SR-6	New user submits a password that breaks the rules, does not match, or uses an invalid/expired credential.	Password or credential check fails.	Nothing stored; message states what failed.
SR-7	User tries to log in before setting a password.	No password set for the account.	Login rejected with a generic authentication-failure message.

5.1.3 Functional Requirements

- **REQ-ONB-001:** The system shall allow only an authenticated Administrator to create accounts, and shall reject any other attempt with an access-denied message and no account created.
- **REQ-ONB-002:** The system shall provide no self-registration function, and shall provide a pre-seeded Administrator account created during initial system setup (handling of its initial credential is specified in Section 6.3).
- **REQ-ONB-003:** The system shall require the Administrator to select exactly one role (Student or Faculty) for each new account, and shall store the selected role with the account.
- **REQ-ONB-004:** For a Student account, the system shall require SRN, name, email and academic/programme details (as defined in Appendix B).
- **REQ-ONB-005:** For a Faculty account, the system shall require Staff ID, name, email and department details (as defined in Appendix B).
- **REQ-ONB-006:** The system shall neither require nor accept a password from the Administrator when creating a Student or Faculty account.
- **REQ-ONB-007:** The system shall trim leading and trailing whitespace from text inputs, and shall reject a request in which a required field is missing or blank, an email is not in a valid format (one "@" with non-empty parts and a "." in the domain), or any value violates the limits in Appendix B.
- **REQ-ONB-008:** When validation fails, the system shall create no account, shall display an error message identifying every invalid or missing field, and shall retain the entered non-password values.
- **REQ-ONB-009:** The system shall ensure that no two accounts share the same SRN, Staff ID, Administrator ID or email address (compared without regard to letter case), including when two requests are submitted simultaneously.
- **REQ-ONB-010:** When a submitted identifier or email duplicates an existing account, the system shall reject the request, leave the existing account unchanged, and display an error message that names the duplicated field and discloses no other data of the existing account.
- **REQ-ONB-011:** The system shall set a successfully created account to Active with no

approval step, and shall record the creation time and the creating Administrator.

- **REQ-ONB-012:** Upon successful account creation, the system shall initiate a secure initial-password setup through which the user sets their own password, and shall never reveal that password to the Administrator or any other user.
- **REQ-ONB-013:** The system shall issue the setup credential as single-use, bound to one account and valid for a limited period (period and delivery mechanism TBD in Section 6.3).
- **REQ-ONB-014:** The system shall reject a setup credential that is used, expired or invalid, shall set no password, and shall inform the user that the credential is not valid.
- **REQ-ONB-015:** The system shall require the password to be entered twice with matching entries, to have at least 8 characters, and to contain at least one uppercase letter, one lowercase letter, one digit and one special character; otherwise it shall not store the password and shall state which rule was not met.
- **REQ-ONB-016:** The system shall store a password only as a salted, adaptive, one-way hash (algorithm defined in Section 6.3), and shall not store, log or display it in plaintext.
- **REQ-ONB-017:** Upon successful password setup, the system shall invalidate the setup credential, shall create no authenticated session, and shall direct the user to log in separately.
- **REQ-ONB-018:** The system shall use the registered email address as the login identifier, and shall reject login for an account whose password is not yet set with the same generic message used for incorrect credentials.
- **REQ-ONB-019:** Upon successful account creation, the system shall show the Administrator a confirmation containing the new account's identifier, name and role, and no password, hash or setup credential.
- **REQ-ONB-020:** The system shall create an account atomically, leaving no partial account if any step fails, and shall show a generic error message that exposes no internal details when an unexpected error occurs.

5.2 Edit Records

5.2.1 Description and Priority

Description. Edit Records lets an authorised user modify an existing student record after it has been created through onboarding (Section 5.1): personal and contact details, programme and section, and account status. Records are never deleted. A student who leaves is set to Inactive instead, which blocks their login while their record, attendance and grades are retained and remain viewable by Administrators. Every edit is written to an audit trail. Corrections to attendance entries are covered in Section 5.3, and amendments to grades in Section 5.4.

Priority: High. Attendance, grading and retrieval all depend on the accuracy of the student record, so uncorrected errors spread to every other feature.

Preconditions. - The student record exists (created through Section 5.1). - The Administrator performing the edit is authenticated. **Inputs.**

Field	Editable	Notes
SRN and system-generated identifiers	No	Fixed once the record is created.
Name	Yes	
Email	Yes	Must remain unique.
Phone number, address	Yes	

Field	Editable	Notes
Programme, section	Yes	
Account status	Yes	Active or Inactive. Setting it to Inactive deactivates the account (login is blocked) and requires confirmation.

Field lengths, data types, mandatory status and the final field list are defined in Appendix B.

Assumptions and dependencies. - Only Administrators may edit student records. What each role may do is defined in Section 5.6. - The Administrator finds the record to edit using retrieval, defined in Section 5.5. - The response-time target is defined in Section 6.1. - Email comparisons ignore letter case and leading/trailing whitespace, as in Section 5.1.

5.2.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Administrator selects "Edit" on a student record found through retrieval.	Administrator authenticated; record exists.	Record displayed in an editable form showing the current values.
SR-2	Administrator submits valid changes to one or more fields.	Role check passes; every field valid; unique fields remain unique; record unchanged by anyone else since it was opened.	All changes saved together; audit entry written; updated record displayed as confirmation.
SR-3	Administrator submits an invalid value (for example a malformed email).	Field validation fails.	Nothing saved; message shown beside each invalid field; other entered values retained.
SR-4	Administrator submits changes to a record that another user saved after it was opened.	Version check finds the record has changed.	Update rejected; user told the record has changed; latest values displayed so the edit can be re-applied.

#	Stimulus (trigger)	System validation	System response
SR-5	Administrator sets a student's account status to Inactive.	Confirmation required and given.	Account deactivated: login blocked and authenticated sessions invalidated; record retained and viewable by Administrators; student excluded from new attendance and grade entry; change logged.
SR-6	A Faculty member or Student attempts to edit a student record.	Role check fails.	Access denied; no record data disclosed; attempt logged.

5.2.3 Functional Requirements

- **REQ-REC-001:** The system shall allow only users with the Administrator role to edit student records.
- **REQ-REC-002:** The system shall allow the following fields to be edited: name, email, phone number, address, programme, section and account status (full field list in Appendix B).
- **REQ-REC-003:** The system shall not allow the student's SRN or any system-generated identifier to be changed.
- **REQ-REC-004:** The system shall validate every submitted field against its type, length, format and mandatory status as defined in Appendix B, and shall reject invalid input with a message that names the field and the problem.
- **REQ-REC-005:** The system shall reject an edit that would make a unique field (such as email) duplicate that of another student record, and shall name the field in conflict.
- **REQ-REC-006:** The system shall apply all changes in a single submission together or not at all, leaving the record exactly as it was and showing an error message if saving fails for any reason.
- **REQ-REC-007:** The system shall detect when a record has been modified by someone else since the user opened it, shall reject the stale update, and shall display the current values.
- **REQ-REC-008:** The system shall write an audit entry for every successful edit containing the user, the date and time, the field changed, and its old and new values, and shall provide no means to modify or delete audit entries.
- **REQ-REC-009:** The system shall not permit a student record to be permanently deleted through the application; removing a student from active use shall be done by setting the account status to Inactive, which deactivates the account as specified in REQ-RBAC-018 and REQ-RBAC-019, after the Administrator confirms the action.
- **REQ-REC-010:** The system shall retain Inactive students' records, attendance and grades in full, shall keep them viewable and retrievable by Administrators, and shall exclude the students from new attendance and grade entry.
- **REQ-REC-011:** The system shall allow an Administrator to reactivate an Inactive student, restoring the student's ability to log in, and shall log the change.
- **REQ-REC-012:** The system shall deny any edit attempt by an unauthorised user without disclosing record contents, and shall log the attempt with the user and time.

- **REQ-REC-013:** The system shall complete a record edit within the response-time target defined in Section 6.1.

5.3 Mark Attendance (with constraints)

5.3.1 Description and Priority

Description. Mark Attendance lets Faculty members record whether each student was present or absent at a scheduled class session, and lets students view their own attendance. Attendance is recorded per student per class session, and only for sessions that have been scheduled. Two constraints apply. A student cannot be marked Present in two class sessions whose scheduled times overlap, and a Faculty member can correct an entry only within a limited edit window after the session. After that window only an Administrator can amend an entry, with a written reason and the change logged (Appendix D). Every entry and change is written to the audit trail.

Priority: High. Attendance is one of the system's core purposes, and the institution reported that attendance is currently entered late and cannot be traced to a person (Appendix D).

Preconditions. - The class session exists in the schedule, for a course offering that the Faculty member teaches. - The student is enrolled in that course offering and is Active. - The user is authenticated.

Inputs.

Data	Provided by	Details
Class session	Faculty member	A scheduled session of a course offering, with its date, start time and end time.
Attendance status	Faculty member	One status per listed student: Present or Absent.
Amendment reason	Administrator	Free text, required for every entry or amendment made by an Administrator.

Edit window. A Faculty member can mark attendance for a session from its scheduled start time until 24 hours after its scheduled end time. The window is measured against the server clock. The length of the window is a team decision, since Appendix D leaves it open.

Assumptions and dependencies. - What each role may do is defined in Section 5.6. A Faculty member's right to mark attendance depends on teaching the course offering. - The schedule of class sessions is assumed to exist. How it reaches the system is TBD, and timetable generation is out of scope (Section 2.6). - Enrolment of students in course offerings is assumed to exist (Section 2.6). - Inactive students are excluded from new attendance entry (REQ-REC-010). - Only the statuses Present and Absent are provided in this version. Other statuses, such as late or excused, are not. - The response-time target is defined in Section 6.1. Marking attendance for a class of up to 60 students in under 2 minutes is covered by REQ-QUA-006.

5.3.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Faculty member opens attendance for a scheduled session of a course they teach, at or after its start time.	Authenticated; teaches the course offering; session is scheduled; start time reached.	Enrolled Active students listed, one entry each, showing the current status where one exists.
SR-2	Faculty member submits a status for every listed student.	Every student has a status; no overlap conflict; records unchanged since opened.	All entries saved together; audit entries written; confirmation shows the number of students marked Present and Absent.
SR-3	Faculty member submits Present for a student who is already Present in an overlapping session.	Overlap check fails.	Nothing from the submission saved; each conflicting student and session is named.
SR-4	Faculty member tries to mark a session that is not scheduled, has not yet started, or belongs to a course they do not teach.	Schedule, time or teaching check fails.	Request rejected with a message stating the reason; a request for a course they do not teach is denied and logged.
SR-5	Faculty member corrects an entry while the edit window is open.	Role, teaching and window checks pass.	Entry updated; old and new status logged.
SR-6	Faculty member corrects an entry after the edit window has closed.	Window check fails.	Entry unchanged; user told that only an Administrator can amend it.
SR-7	Administrator enters or amends an entry and supplies a reason.	Role is Administrator; reason supplied; overlap check passes.	Entry saved and flagged as amended; old status, new status and reason logged.
SR-8	Faculty member or Administrator submits changes to attendance that another user saved after it was opened.	Version check finds the data has changed.	Update rejected; current values displayed so the edit can be re-applied.

#	Stimulus (trigger)	System validation	System response
SR-9	Student opens their own attendance.	Authenticated; role is Student.	Own attendance shown read-only for each course, with a summary of sessions marked, sessions present and the attendance percentage.
SR-10	A Student attempts to mark or change attendance, or a Faculty member requests attendance of a course they do not teach.	Role or teaching check fails.	Access denied; no data disclosed; attempt logged.

5.3.3 Functional Requirements

- REQ-ATT-1:
- REQ-ATT-2:

5.4 Mark Grades (with rubric)

5.4.1 Description and Priority

Description. Mark Grades lets faculty record marks for students against a grading rubric, and lets students see their published results. A rubric is a set of weighted components (for example assignments, internal tests and a final exam). It is specific to each course rather than a fixed scheme for the whole institution. Each course has one **anchor faculty**: a faculty member teaching that course, designated by an Administrator. Only the anchor faculty can create or alter the course's rubric, which keeps the grading scheme consistent for every student in the course. Other faculty teaching the course can view the rubric and enter marks against it but cannot change it. Administrators can reassign the anchor faculty but cannot edit rubrics themselves. Marks are entered as drafts and become visible to students only when the anchor faculty publishes them. Changes after publication are restricted and logged.

Priority: High. Grading is one of the system's core purposes, and the rubric rules directly affect the correctness of every student's result.

Preconditions. - The course exists and has at least one faculty member assigned to teach it. - The user is authenticated. - To enter marks, the course's rubric for that term has been defined, and the students are enrolled in the course. **Inputs.**

Data	Provided by	Details
Anchor faculty assignment	Administrator	The course, and the faculty member designated as its anchor.

Data	Provided by	Details
Rubric component	Anchor faculty	Name (unique within the rubric), maximum mark greater than zero, weight greater than zero.
Mark	Faculty teaching the course	Student, rubric component, and a number from 0 to the component's maximum with at most two decimal places.
Amendment reason	Anchor faculty	Free text; required for every change made after publication.

Assumptions and dependencies. - What each role may do is defined in Section 5.6. Being the anchor faculty is an additional permission that applies to one course only. - Courses, terms and the enrolment of students in courses are assumed to exist. How they are created is TBD. - Marks and rubric changes are written to the same audit trail used in Section 5.2. - The institution's letter-grade bands are TBD (see REQ-GRD-023). - The response-time target is defined in Section 6.1.

5.4.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Anchor faculty submits a rubric whose weights total 100%.	User is the course's anchor faculty; component names unique; maximum marks and weights greater than zero; weights total 100%.	Rubric saved; change logged; confirmation displayed.
SR-2	Anchor faculty submits a rubric whose weights do not total 100%.	Weight total check fails.	Nothing saved; current total displayed so it can be corrected.
SR-3	A faculty member who is not the course's anchor opens the rubric page, or sends a change request directly.	Anchor check fails for any change.	Rubric displayed read-only with no editing controls; a direct change request is denied and logged.
SR-4	Faculty member opens the gradebook for a course they teach.	User teaches the course; rubric exists.	Enrolled active students listed, with one column per rubric component.

#	Stimulus (trigger)	System validation	System response
SR-5	Faculty member submits valid marks.	Every mark is a number within its component's range; faculty teaches the course; records unchanged since opened.	Marks saved as drafts; each student's weighted total recalculated and displayed.
SR-6	Faculty member submits a mark that is outside the allowed range or not a number.	Range or number check fails.	Nothing from the submission saved; each invalid entry listed with its student, component and allowed range.
SR-7	Anchor faculty selects "Publish".	User is the course's anchor faculty; all enrolled active students checked for missing marks.	If marks are missing, the students are listed and confirmation requested. On confirmation, grades become visible to students.
SR-8	Student opens their results.	Student authenticated; grades published.	Only that student's grades shown, with each component's mark, maximum and weight, and the weighted total.
SR-9	Anchor faculty corrects a mark after publication.	User is the anchor faculty; written reason supplied.	Mark updated and flagged as amended; old value, new value and reason logged; student sees the updated value.

5.4.3 Functional Requirements

- **REQ-GRD-001:** The system shall allow an Administrator to designate exactly one anchor faculty for each course, who must be a faculty member assigned to teach that course.
- **REQ-GRD-002:** The system shall allow only the anchor faculty of a course to create, modify or delete that course's rubric, and shall give other faculty of the same course and Administrators read-only access to it.
- **REQ-GRD-003:** The system shall define a rubric as one or more components, each having a name that is unique within the rubric, a maximum mark greater than zero, and a weight greater than zero.
- **REQ-GRD-004:** The system shall accept a rubric only if the component weights add up to exactly 100%, and shall otherwise reject it and display the current total.
- **REQ-GRD-005:** The system shall maintain a separate rubric for each course in each term,

and shall allow the anchor faculty to copy a previous term's rubric as the starting point for a new one.

- **REQ-GRD-006:** The system shall allow the rubric to be modified freely while no marks have been recorded against it; if marks have been recorded but not published, the system shall warn the anchor faculty, require confirmation before saving and recalculate all affected totals; once grades are published, the system shall not allow the rubric to be changed.
- **REQ-GRD-007:** The system shall record every rubric creation and modification in the audit trail with the user, date and time, and the old and new values.
- **REQ-GRD-008:** The system shall allow an Administrator to reassign the anchor faculty of a course to another faculty member teaching it, with the change taking effect immediately, removing the previous anchor's rubric rights and being logged.
- **REQ-GRD-009:** The system shall not allow marks to be entered for a course until its rubric exists.
- **REQ-GRD-010:** The system shall allow a faculty member to enter and edit marks only for active students enrolled in courses that the faculty member teaches.
- **REQ-GRD-011:** The system shall accept a mark for a rubric component only if it is a number between 0 and that component's maximum mark inclusive, with at most two decimal places.
- **REQ-GRD-012:** The system shall reject an invalid mark with a message naming the student, the component and the allowed range, and when several marks are submitted together and any is invalid, shall report all invalid entries and save none of the submission.
- **REQ-GRD-013:** The system shall distinguish a mark of zero from a mark not yet entered, and shall display a student's total as "Incomplete" until every component has a mark.
- **REQ-GRD-014:** The system shall calculate each student's weighted total as the sum, over all components, of (mark $\div$ maximum mark) $\times$ weight, rounded to two decimal places, and shall recalculate it automatically whenever a mark or the rubric changes.
- **REQ-GRD-015:** The system shall detect and reject a mark update based on stale data, in the same way as REQ-REC-007, and shall display the current value.
- **REQ-GRD-016:** The system shall record every mark entry and change in the audit trail with the user, date and time, student, component, and old and new values.
- **REQ-GRD-017:** The system shall deny any attempt by a user to view or change marks for a course they do not teach, without disclosing any marks, and shall log the attempt.
- **REQ-GRD-018:** The system shall allow only the anchor faculty to publish a course's grades, and if any enrolled active student has missing marks, shall list them and require confirmation before publishing.
- **REQ-GRD-019:** The system shall make unpublished marks visible only to the faculty teaching the course and to Administrators.
- **REQ-GRD-020:** The system shall allow a student to view only their own published grades, showing for each component the mark, maximum and weight, together with the weighted total.
- **REQ-GRD-021:** After publication, the system shall allow only the anchor faculty to amend a mark, shall require a written reason, shall flag the grade as amended, and shall record the old value, new value and reason in the audit trail.
- **REQ-GRD-022:** The system shall retain the marks of students who later become Inactive.
- **REQ-GRD-023:** The system shall convert a weighted total to a letter grade according to the institution's grade bands (TBD: the bands have not yet been confirmed and will be added once the grading policy is fixed).
- **REQ-GRD-024:** The system shall complete a mark entry or retrieval within the response-

time target defined in Section 6.1.

5.5 Retrieval

5.5.1 Description and Priority

TODO

5.5.2 Stimulus/Response Sequences

TODO

5.5.3 Functional Requirements

- REQ-RET-1:
- REQ-RET-2:

5.6 Role-Based Access

5.6.1 Description and Priority

Description. Role-Based Access Control (RBAC) decides which functions and records each authenticated user may use. Every account has exactly one role: Administrator, Faculty or Student. The role is assigned at account creation (Section 5.1), and only an Administrator can change it, and only between Student and Faculty. Administrator accounts are pre-seeded only (Section 5.1), so the Administrator role can neither be assigned nor removed through the application. Some permissions also depend on a user's relationship to a course, such as being its anchor faculty (Section 5.4). Permissions are enforced in both the frontend and the backend, and the backend is the authoritative enforcement layer. Anything not explicitly permitted to a role is denied. Authentication (Section 5.1 and Section 6.3) establishes who the user is, and authorisation (this section) decides what that user may do.

Priority: High. Features 5.2 to 5.5 read and modify student data and depend on correct authorisation.

Roles and permissions.

Role	Permitted	Scope boundary	Not permitted
Administrator	Create, update and deactivate Student and Faculty accounts; change roles between Student and Faculty; view and manage student records (Section 5.2); view all attendance and grades, and amend attendance as allowed by Section 5.3; designate and reassign the anchor faculty of each course (Section 5.4); view all Faculty and Student records	All relevant records	Create, alter or delete rubrics; enter or amend marks
Faculty	View students enrolled in courses they teach; mark and edit attendance and enter and edit marks for those students; view the academic records of those students; as the anchor faculty of a course, also create and alter its rubric, publish its grades and amend published marks (Section 5.4)	Courses they teach, and the students enrolled in them	Students outside those courses; account and role management; editing student records; changing the rubric of a course for which they are not the anchor faculty
Student	View own profile, attendance, published grades and academic records	Own records only	Other students' records; modifying own grades or attendance

Assumptions and dependencies. - Attendance rules (Section 5.3) and grading rules (Section 5.4) apply in addition to RBAC. - The authenticated session mechanism, audit-trail retention and who may view the audit trail are defined in Section 6.3 (TBD). - A Faculty member's scope is determined by the courses they teach and the students enrolled in them. How courses, teaching assignments and enrolments are created is TBD (see Section 5.4). RBAC only requires that they exist. - TBD in this version: whether an Administrator may deactivate their own or the last remaining Administrator account (Administrator accounts cannot be created through the application, so this could lock everyone out); and how records and teaching assignments are handled when an account's role changes between Student and Faculty.

5.6.2 Stimulus/Response Sequences

#	Stimulus (trigger)	System validation	System response
SR-1	Administrator requests an Administrator-permitted function (e.g. view any record, designate an anchor faculty, deactivate an account).	Authenticated session valid; account Active; role is Administrator.	Request processed and result shown.
SR-2	Faculty member requests the record, or an attendance or grade function, for a student enrolled in a course they teach.	Authenticated session valid; role is Faculty; student is enrolled in a course taught by this Faculty member.	Request processed, subject to Sections 5.3 and 5.4.
SR-3	Faculty member requests a student who is not enrolled in any course they teach.	Student not enrolled in a course taught by this Faculty member.	"Access Denied/Unauthorized"; no data returned; attempt logged.
SR-4	Student requests their own profile, attendance, grades or academic records.	Authenticated session valid; role is Student; record belongs to this Student.	Record displayed read-only.
SR-5	Student requests another student's record, or tries to modify their own grades or attendance.	Record not owned, or modification not permitted for Student.	"Access Denied/Unauthorized"; data unchanged; attempt logged.
SR-6	Any user requests a function not permitted for their role (e.g. Faculty member changes a role).	Function not in the role's permission set.	"Access Denied/Unauthorized"; no change; attempt logged.
SR-7	Administrator changes a user's role.	Requester is Administrator; account exists; account is not an Administrator account; new role is Student or Faculty.	Role updated and effective immediately; change recorded; confirmation shown. A request involving the Administrator role is rejected with an error message.

#	Stimulus (trigger)	System validation	System response
SR-8	Inactive user tries to log in or uses an existing authenticated session.	Account status is Inactive.	Request rejected; no data returned; no new authenticated session created.
SR-9	Unauthenticated user, or user with an expired/invalid authenticated session, requests a protected function.	Authentication check fails.	Request rejected; user directed to log in; no data returned.

5.6.3 Functional Requirements

- **REQ-RBAC-001:** The system shall support exactly three roles (Administrator, Faculty and Student) and shall associate every account with exactly one role at all times.
- **REQ-RBAC-002:** The system shall deny any function or record access that is not explicitly permitted to the requesting user's role.
- **REQ-RBAC-003:** The backend shall verify authentication and authorisation (role and record scope) on every protected request before processing it, regardless of any frontend check.
- **REQ-RBAC-004:** The frontend shall display only the functions permitted to the logged-in user's role.
- **REQ-RBAC-005:** The system shall associate the authenticated user's role with the authenticated session, check it on every protected request, and not accept a client-supplied role, identity or ownership claim in its place.
- **REQ-RBAC-006:** The system shall deny a protected request whose authenticated session is missing, expired, invalid or inconsistent with the stored account, and for an unauthenticated user shall direct the user to log in without returning protected data.
- **REQ-RBAC-007:** The system shall allow an Administrator to create, update and deactivate Student and Faculty accounts, and to change roles between Student and Faculty (REQ-RBAC-016).
- **REQ-RBAC-008:** The system shall allow an Administrator to view and manage the records of all students (Section 5.2), to view the attendance and grades of all students and amend attendance as allowed by Section 5.3, to designate and reassign the anchor faculty of a course (Section 5.4), and to view all Faculty records, and shall not allow an Administrator to create, alter or delete rubrics or to enter or amend marks.
- **REQ-RBAC-009:** The system shall allow a Faculty member to view the students enrolled in the courses they teach, to mark and edit attendance and enter and edit marks for those students (subject to Sections 5.3 and 5.4), and to view those students' relevant academic records.
- **REQ-RBAC-010:** The system shall deny a Faculty member any read or write access to a student who is not enrolled in a course that the Faculty member teaches, and shall evaluate this on every request so that a change takes effect on the next request (how teaching assignments and enrolments are created is TBD).
- **REQ-RBAC-011:** The system shall deny a Faculty member the creation, update and deactivation of accounts, the assignment or change of roles, and the editing of student records.

- **REQ-RBAC-012:** The system shall allow a Student to view their own profile, attendance, published grades and academic records.
- **REQ-RBAC-013:** The system shall deny a Student any access to another student's records, any modification of their own grades or attendance, and any Faculty or Administrator function.
- **REQ-RBAC-014:** When a request is denied for lack of authorisation, the system shall return a clear "Access Denied/Unauthorized" message, process no part of the request, leave data unchanged, and give the same response for a nonexistent record as for an unauthorised one.
- **REQ-RBAC-015:** When an authorisation check cannot be completed because of an internal error, the system shall deny the request and show a generic error message that exposes no internal details.
- **REQ-RBAC-016:** The system shall allow only an Administrator to change an account's role, shall accept only Student or Faculty as the new role, shall not allow any account to be given the Administrator role or an Administrator account's role to be changed (Administrator accounts are pre-seeded only), and shall reject a role change for a nonexistent account with an error message.
- **REQ-RBAC-017:** The system shall apply a role change immediately, so that every later protected request (including from existing authenticated sessions) is authorised against the new role, shall keep exactly one role on the account, and shall record the affected account, previous role, new role, Administrator and time.
- **REQ-RBAC-018:** The system shall allow only an Administrator to deactivate or reactivate an account, shall invalidate all active authenticated sessions of that account immediately on deactivation, and shall retain all details of the account and keep them viewable by Administrators.
- **REQ-RBAC-019:** The system shall reject login by an Inactive account without creating an authenticated session, and shall deny every protected request made with an authenticated session of an Inactive account.
- **REQ-RBAC-020:** The system shall record in the audit trail every denied protected request and every failed authentication or session validation on a protected request (recording time, user identity and role if known, requested function or record, and outcome), shall not record passwords, hashes or authenticated-session secrets in it, and shall allow no role to modify or delete audit-trail entries through application functions.

6 Other Nonfunctional Requirements

6.1 Performance Requirements

TODO

6.2 Safety Requirements

The Student Record System does not control any physical equipment, so it poses no physical safety hazard and no safety certification is required. The harm it could cause is to the institution's data and to the people the data describes: records that are lost or corrupted, grades that are calculated wrongly, or consequential actions taken by mistake. The requirements below guard against these. Protection of personal data from unauthorised access is covered in Section 6.3.

Data loss and recovery

- **REQ-SAF-001:** The system shall support a full backup of the database at least once every 24 hours, and the 14 most recent backups shall be retained.
- **REQ-SAF-002:** Backups shall be stored on storage separate from the storage that holds the live database.
- **REQ-SAF-003:** The project shall provide a documented procedure for restoring the database from a backup, and the procedure shall be tested successfully before release. A restored system shall be in a consistent state, with the audit trail intact up to the time of the backup.
- **REQ-SAF-004:** After a restore, no more than 24 hours of changes shall be lost. **Data integrity**
- **REQ-SAF-005:** The system shall run every operation that changes several records together as a single transaction, so that it completes in full or leaves no change (see REQ-REC-006, REQ-ONB-020 and Section 3.2).
- **REQ-SAF-006:** The database shall itself enforce uniqueness and referential integrity, so that an application defect cannot create orphaned data, such as a mark for a student or rubric component that does not exist.
- **REQ-SAF-007:** The system shall calculate marks and weighted totals using exact decimal arithmetic rather than floating-point arithmetic, so that rounding errors cannot change a student's result.
- **REQ-SAF-008:** When the database is unavailable, the system shall reject any request that would change data, shall show a clear message saying so, shall not apply any part of the request, and shall resume normal operation automatically when the database returns. **Preventing mistakes**
- **REQ-SAF-009:** The system shall require explicit confirmation before it deactivates an account, publishes a course's grades, amends a published mark, or changes a rubric after marks have been recorded (see REQ-REC-009, REQ-GRD-006, REQ-GRD-018 and REQ-GRD-021).
- **REQ-SAF-010:** Development and test environments shall use synthetic data only, and shall never hold real student data.

6.3 Security Requirements

Security requirements for authentication, password protection, session handling, authorization enforcement, and protection of student data. They complete the items that Sections 5.1 and 5.6 defer to this section: hashing algorithm, session mechanism, setup-credential rules, and audit-log rules. Functional behaviour of onboarding and role-based access is in Sections 5.1 and 5.6 and is not repeated here. The numeric values below (for example timeouts and the lockout threshold) are team decisions for this project; the other numbers are reasonable defaults for a student record system.

Scope note. Password change and password reset are not part of this version. Email is used only to deliver the initial-password setup credential (Section 5.1).

Authentication and password protection

- **REQ-SEC-001:** The system shall authenticate a user only by verifying the submitted password against the stored hash of the account matching the submitted registered email, and shall create a session only when verification succeeds for an Active account whose password has been set.
- **REQ-SEC-002:** The system shall return the same generic authentication-failure message, with no indication of which part was wrong, for an unknown email, an incorrect password, a deactivated account, a locked account, and an account whose password is not yet set.
- **REQ-SEC-003:** The system shall hash passwords with bcrypt using a unique random salt per password and a cost factor of at least 10, and shall verify passwords only by comparing against this hash.
- **REQ-SEC-004:** The system shall never return, display, log or export a password or password hash in any response, screen, report or log entry.
- **REQ-SEC-005:** The system shall lock login for an account for 15 minutes after 5 consecutive failed login attempts, shall reset the failure count after a successful login, and shall show the generic message of REQ-SEC-002 while the lock is active.
- **REQ-SEC-006:** The system shall generate each initial-password setup credential with a cryptographically secure random generator and at least 128 bits of randomness, deliver it to the user's registered email address, make it valid for 24 hours and for one use only, and invalidate it if the account is deactivated.
- **REQ-SEC-007:** The system shall take the pre-seeded Admin's initial credential from the initial system setup, apply the password rules of REQ-ONB-015 and the hashing of REQ-SEC-003 to it, and shall not hard-code it in source code or commit it to the project repository.

Session and token security

- **REQ-SEC-008:** The system shall keep session state on the server, shall give the client only an opaque session identifier generated with a cryptographically secure random generator and containing no role or personal data, shall issue a new identifier at every successful login, and shall obtain the user's role and account status from the server's stored data on every protected request.
- **REQ-SEC-009:** The system shall expire a session after 30 minutes without activity or 8 hours after login, whichever comes first, and shall invalidate the session on the server immediately when the user logs out; an expired or logged-out session shall be rejected on every protected request.
- **REQ-SEC-010:** The system shall carry all client-server communication over an encrypted connection, shall reject or redirect unencrypted requests, and shall send the session identifier only over that connection and in a form not readable by client-side scripts.

Authorization enforcement and record confidentiality

- **REQ-SEC-011:** The system shall perform all authorization checks in the backend in one common enforcement layer through which every protected request passes, so that a protected function is unreachable without passing the checks of REQ-RBAC-003, and a newly added protected function is denied until a role permission is defined for it.
- **REQ-SEC-012:** The system shall apply the access boundaries of Section 5.6 to every way of obtaining student data, including retrieval, search, lists, reports and exports, so that a result contains only records the requesting user is authorized to see.

Input validation and protection against common attacks

- **REQ-SEC-013:** The system shall validate the type, length and format of every user-supplied

input in the backend, regardless of any frontend validation, and shall reject invalid input without processing it.
- **REQ-SEC-014:** The system shall never interpret user-supplied input as part of a query or command (it shall be passed to the data store only as bound parameters), and shall encode user-supplied data when displaying it so that it cannot run as script in the user's browser.
- **REQ-SEC-015:** The system shall reject any state-changing request that does not carry a valid anti-forgery token tied to the user's session.

Secure error handling and audit logging

- **REQ-SEC-016:** In every feature, the system shall show only generic error messages that contain no stack traces, file paths, query text, or other internal details, and shall record such details only in server-side logs.
- **REQ-SEC-017:** In addition to the events required by REQ-RBAC-020, the system shall log successful and failed logins, account lockouts, account creations, role changes and account deactivations, each with time, acting user, affected account and outcome; the log shall be retained for at least 1 year and shall be viewable only by Admins, read-only.

Data protection and security validation

- **REQ-SEC-018:** The system shall use student and user data only within the Student Record System, shall not disclose it to third parties, and shall not include student records in any email.
- **REQ-SEC-019:** The project's continuous integration shall run static code analysis, and a release shall not be made while high-severity security findings remain unresolved.

6.4 Software Quality Attributes

The quality attributes below matter most to the institution and to the development team. They are listed in order of importance when two of them conflict:

1. **Correctness and data protection.** A wrong grade or a lost record does more harm than a slow page.
2. **Reliability.** Faculty must be able to rely on the system at the moments they need it, such as the start and end of class periods.
3. **Usability for Faculty,** who enter most of the data, over usability for occasional users.
4. **Maintainability and testability,** which the institution's IT team and the course guidelines both require.
5. **Performance beyond the targets in Section 6.1,** which is the lowest priority once the targets are met.

Correctness

- **REQ-QUA-001:** A release shall pass all of its automated tests and shall have no open defect of high or critical severity.

Reliability and availability

- **REQ-QUA-002:** The system shall be available at least 99% of the time in each calendar month during the institution's working hours, excluding planned maintenance announced at least 24 hours in advance.
- **REQ-QUA-003:** A change that the system has confirmed to the user shall not be lost, except through restoring a backup (REQ-SAF-004).

- **REQ-QUA-004:** If a container stops unexpectedly, the system shall restart it automatically and resume service without manual intervention.

Usability

- **REQ-QUA-005:** At least 90% of first-time users in usability testing shall be able to set their password and log in for the first time, without assistance, in under 5 minutes.
- **REQ-QUA-006:** A Faculty member shall be able to mark attendance for a class of up to 60 students in under 2 minutes.
- **REQ-QUA-007:** Every error message shall say what went wrong and, where possible, how to correct it, and shall expose no internal details (see REQ-RBAC-015).
- **REQ-QUA-008:** All functions shall be operable using the keyboard alone, and all text shall meet the WCAG 2.1 level AA colour-contrast requirement.

Maintainability and debuggability

- **REQ-QUA-009:** The software shall be organised in layers that separate request handling, business logic and data access, and each function in Section 5 shall be implemented in its own module.
- **REQ-QUA-010:** The system shall not hard-code configuration such as database addresses, credentials or email settings. They shall be supplied through environment variables or configuration files.
- **REQ-QUA-011:** The backend shall log errors with the time, a request identifier, the user identifier and the function involved, so that a failure can be traced without reproducing it. The log shall contain no passwords, session secrets or personal data other than identifiers.
- **REQ-QUA-012:** Static code analysis shall report no blocker or critical issues at release (see REQ-SEC-019).
- **REQ-QUA-013:** Every API endpoint shall be documented, with its purpose, inputs, outputs and error responses.

Testability

- **REQ-QUA-014:** Automated unit tests shall cover at least 80% of backend lines and at least 70% of backend branches, and the coverage shall be reported on every run of the CI pipeline.
- **REQ-QUA-015:** Every functional requirement in Section 5 shall be traced in Appendix C to at least one test case.

Portability, extensibility and scalability

- **REQ-QUA-016:** The whole system shall start on any Linux host that has Docker and Docker Compose installed, using one command, with no manual steps other than supplying configuration.
- **REQ-QUA-017:** A new function shall be addable without changing the existing Section 5 modules, other than registering the new function with the application.
- **REQ-QUA-018:** The system shall continue to meet the response-time targets in Section 6.1 with four years of accumulated data, which is about 20 million attendance records.

6.5 Business Rules

These are the operating principles of the system: who may do what, and under which circumstances. They are not functional requirements in themselves, but each is enforced by the requirements named

in the Source column. The final group states the domain rules of the academic setting that the system models.

Accounts and roles

ID	Rule	Source
BR-001	Every user holds exactly one role: Administrator, Faculty or Student.	5.6
BR-002	Accounts are created only by an Administrator. There is no self-registration. Administrator accounts are pre-seeded during setup and cannot be created or assigned through the application.	5.1, 5.6
BR-003	Each user sets their own password. No one else, including an Administrator, ever sets or knows it.	5.1
BR-004	Every SRN, Staff ID, Administrator ID and email address is unique across all accounts.	5.1
BR-005	Only an Administrator can change a role, and only between Student and Faculty.	5.6
BR-006	Records are never deleted. A user who leaves is set to Inactive, which blocks their login while their details are retained and remain viewable by Administrators.	5.2, 5.6

Access to records

ID	Rule	Source
BR-007	A Faculty member can act only on the courses they teach and the students enrolled in them.	5.6
BR-008	A Student can view only their own profile, attendance, published grades and academic records, and cannot change any of them.	5.6
BR-009	Only an Administrator can edit student records.	5.2
BR-010	Anything not explicitly permitted to a role is denied.	5.6

Attendance

ID	Rule	Source
BR-011	Attendance can be marked only for a scheduled class session.	5.3
BR-012	A student cannot be marked present in two overlapping class sessions.	5.3
BR-013	A Faculty member can correct an attendance entry only within the edit window. After it closes, only an Administrator can amend the entry, and the change is logged.	5.3

Grading

ID	Rule	Source
BR-014	Each course offering has exactly one anchor faculty, designated by an Administrator from the faculty teaching the course.	5.4
BR-015	Only the anchor faculty can create or alter a course's rubric. Administrators and other faculty cannot.	5.4
BR-016	The weights of a rubric's components add up to exactly 100%, and each mark lies between 0 and its component's maximum mark.	5.4
BR-017	Grades are visible to students only after the anchor faculty publishes them. After publication the rubric is locked, and only the anchor faculty can amend a mark, with a recorded reason.	5.4

Traceability and data protection

ID	Rule	Source
BR-018	Every change to a student record, rubric, mark, attendance entry or role is recorded in the audit trail with who made it and when, and audit-trail entries cannot be altered.	5.2, 5.4, 5.6, 6.3
BR-019	Personal data is handled in line with the Digital Personal Data Protection Act, 2023, and is visible only to the roles permitted to see it.	6.3, 7.2

Domain rules

ID	Rule	Source
BR-020	A student's weighted total for a course offering is the sum, over all rubric components, of (mark ÷ maximum mark) × weight.	5.4
BR-021	A rubric belongs to a course offering, so the same course can have a different rubric in each term.	5.4
BR-022	Conversion of a weighted total to a letter grade follows the institution's grade bands (TBD, Section 5.4).	5.4
BR-023	A student's attendance and grades belong to a course offering, which is a course in a particular term.	5.3, 5.4

7 Other Requirements

This section covers requirements that do not belong to a single function or to the other nonfunctional categories: the database, legal obligations, localisation, reuse, and what must be delivered with the system.

7.1 Database Requirements

- **REQ-OTH-001:** The system shall store all persistent data in a single MySQL relational database (see Sections 2.5 and 3.2).

- **REQ-OTH-002:** The database schema shall be normalised to at least third normal form, except where a departure is documented in the design document with its reason.
- **REQ-OTH-003:** The database shall use a UTF-8 character set that supports the full Unicode range (`utf8mb4` in MySQL), so that names in any script are stored and displayed correctly.
- **REQ-OTH-004:** The database shall compare emails and identifiers without regard to letter case, consistent with REQ-ONB-009.
- **REQ-OTH-005:** Every change to the database schema shall be made through a versioned migration script kept in the project repository. The schema shall not be changed by hand.
- **REQ-OTH-006:** The system shall store all timestamps in UTC and display them in Indian Standard Time (IST).

7.2 Legal and Regulatory Requirements

Student records are personal data. The institution is responsible for its obligations under the Digital Personal Data Protection Act, 2023, and the exact obligations are to be confirmed by the institution's legal advisers. The system shall support the institution in meeting them as follows.

- **REQ-OTH-007:** The system shall collect only the personal data fields defined in Appendix B.
- **REQ-OTH-008:** The system shall let the institution retrieve all the data it holds about a student (Section 5.5) and correct it (Section 5.2).
- **REQ-OTH-009:** The system shall keep student data only for the retention period set by the university's records policy (TBD). The disposal of records after that period is carried out by the institution outside the application, and the procedure is TBD.

7.3 Internationalisation and Localisation

- **REQ-OTH-010:** The user interface shall be in English. User-visible text shall be kept separate from program logic, so that another language can be added later without changing the logic.
- **REQ-OTH-011:** The system shall display dates in the format DD-MM-YYYY and times on a 24-hour clock.

7.4 Reuse

- **REQ-OTH-012:** The system shall use established open-source libraries for security-critical functions, namely password hashing, session handling and input escaping, and shall not implement its own.
- **REQ-OTH-013:** No components are reused from other projects. The modules shall be designed so that they can be reused by the deferred features listed in Section 1.3.

7.5 Initial Setup and Data

- **REQ-OTH-014:** The system shall come with a documented setup procedure that creates the pre-seeded Administrator account (the handling of its initial credential is defined in Section 6.3).
- **REQ-OTH-015:** The initial release shall assume that accounts and student records are entered through the application. Bulk import of existing spreadsheet data is not part of this release (TBD whether it will be added later).

7.6 Documentation and Delivery

- **REQ-OTH-016:** The system shall be delivered with a deployment guide that includes the backup and restore procedure (REQ-SAF-003), a short user guide for each of the three roles, the API documentation (REQ-QUA-013), and a maintenance plan.
- **REQ-OTH-017:** All source code, scripts, configuration and documentation shall be kept in the project's Git repository.

Appendix A: Glossary

This glossary defines the terms and abbreviations used in this SRS. Terms are grouped by topic and listed alphabetically within each group. The role names defined in A.1 are used consistently throughout the document.

A.1 Users and Roles

Term	Definition
Administrator	A user responsible for creating accounts, changing roles between Student and Faculty, deactivating and reactivating accounts, maintaining student records, designating the anchor faculty of each course, and amending attendance after the edit window has closed. Administrators can view rubrics and marks but cannot edit rubrics or enter or amend marks. Identified by an Administrator ID.
Anchor Faculty	The one faculty member, among those teaching a course, who has been designated by an Administrator as responsible for that course's rubric. Only the anchor faculty can create or alter the rubric, publish the course's grades, and amend published grades.
Faculty	A user who teaches one or more courses. Faculty mark attendance and enter grades only for courses they teach. Identified by a Staff ID.
Pre-seeded Administrator	An Administrator account created during initial system setup. Administrator accounts exist only this way: they cannot be created, assigned or removed through the application.
Role	The category of a user account (Administrator, Faculty or Student), which determines what the user may see and do.
Role-Based Access Control (RBAC)	The mechanism by which the system grants or denies each action according to the user's role and, where relevant, their relationship to a course (for example, being its anchor faculty).
Student	A user enrolled at the institution. A student can view only their own record, attendance and published grades. Identified by an SRN.

A.2 Accounts and Records

Term	Definition
Account	The login identity of a user, consisting of a registered email address, a role and a password.

Term	Definition
Account Status	Whether an account is Active or Inactive. An Inactive account cannot log in and has no valid authenticated sessions, but its record and all details are retained and remain viewable by Administrators. Only an Administrator can deactivate or reactivate an account.
Administrator ID	The unique identifier of an Administrator.
Audit Trail	The append-only log of changes made to student records, rubrics, marks, attendance and roles, and of denied requests and failed authentications. Each entry records who, when and what, including the old and new values for changes. It cannot be modified or deleted through the application, and it never contains passwords, hashes or authenticated-session secrets.
Active	The status of an account that is in use. New accounts are created as Active.
Inactive	The status of a deactivated account. The user cannot log in and their authenticated sessions are invalidated. For a student, the student is also excluded from new attendance and grade entry. The record, attendance, grades and all other details are retained and remain viewable by Administrators. Records are never deleted.
Onboarding	The process by which an Administrator creates a Student or Faculty account and assigns its role. Self-registration is not supported.
Record Version	A number stored with each editable record and changed on every update, used to detect stale updates.
Retrieval	Searching for and viewing records, attendance or grades, subject to the user's role.
Setup Credential	A single-use, time-limited token issued when an account is created, which allows the new user to set their own password.
SRN	Student Registration Number: the unique identifier of a student. It cannot be changed after the record is created.
Staff ID	The unique identifier of a Faculty member.
Stale Update	An attempt to save changes to a record that has been modified by someone else since the user opened it. The system rejects such updates.
Student Record	The stored details about a student: SRN, name, contact details, programme, section and account status.

A.3 Academic Structure

Term	Definition
Course	A subject taught at the institution.
Course Offering	A course as taught in a particular term. Rubrics, enrolments and grades belong to a course offering.
Department	The academic department to which a Faculty member belongs.
Enrolment	The link between a student and a course offering. It determines which students a faculty member can mark attendance and grades for.
Programme	The degree or course of study that a student is following.
Section	A group of students within a programme who attend classes together.
Term	A defined period of teaching, such as a semester.

A.4 Attendance

Term	Definition
Attendance Record	The record of one student's attendance status for one class session.
Class Session	A single scheduled meeting of a course offering at which attendance can be marked. This is distinct from an authenticated session (A.6).
Edit Window	The limited period after a class session during which a faculty member can correct an attendance entry. After it closes, only an Administrator can amend the entry, and the change is logged. Its length is specified in Section 5.3.
Overlapping Sessions	Class sessions whose times overlap. A student cannot be marked present in two overlapping sessions.

A.5 Grading

Term	Definition
Amended Grade	A published mark that was changed by the anchor faculty after publication. It is flagged as amended and the reason is recorded in the audit trail.
Draft Grade	A mark that has been entered but not yet published. Draft grades are visible only to the faculty teaching the course and to Administrators.
Grade Bands	The ranges of weighted total that correspond to each letter grade. To be confirmed once the institution's grading policy is fixed.
Incomplete	The state of a student's weighted total while one or more rubric components have no mark entered. A mark of zero is different from a mark not yet entered.
Mark	The score awarded to one student for one rubric component. It must lie between 0 and that component's maximum mark.
Maximum Mark	The highest mark possible for a rubric component.
Publish	The action by which the anchor faculty makes a course's grades visible to its students. After publication, the rubric is locked and changes to marks require a recorded reason.
Published Grade	A grade that has been made visible to students by publication.
Rubric	The set of weighted components against which students in a course offering are graded. It is defined per course offering and altered only by the anchor faculty.
Rubric Component	One assessed item within a rubric, such as an assignment, internal test or final exam, with a name, a maximum mark and a weight.
Weight	The percentage that a rubric component contributes to the weighted total. The weights of all components in a rubric must add up to 100%.
Weighted Total	A student's overall result for a course offering, calculated as the sum over all components of (mark ÷ maximum mark) × weight.

A.6 Security and Technical Terms

Term	Definition
API	Application Programming Interface: the set of requests through which the frontend communicates with the backend.
Anti-Forgery Token	A secret value sent with each state-changing request, so that the backend can reject requests forged by another website. Also known as a CSRF token.
Atomic Operation	An operation that either completes in full or has no effect at all, leaving no partial changes.
Authentication	Verifying that a user is who they claim to be, by means of their email address and password.
Authenticated Session	The period after a successful login during which the system accepts a user's requests. This is distinct from a class session (A.4). The mechanism is defined in Section 6.3.
Authorisation	Deciding whether an authenticated user is permitted to perform a particular action.
Availability	The proportion of time during which the system is usable.
Backend	The server-side application, built with Flask, that enforces access control, validation and business rules and is the only component that accesses the database.
Backup	A complete copy of the database, kept separately so that the system can be restored after data loss.
bcrypt	The password-hashing algorithm used by the system (Section 6.3).
Bound Parameter	A value passed to the database separately from the text of a query, so that user input is never interpreted as part of the query. This prevents SQL injection.
Cloud Deployment	Running the system on a third-party provider's infrastructure (for example AWS, GCP, Azure or Firebase). Considered and not chosen (Appendix D).
Code Coverage	The proportion of the code that is executed by automated tests. Line coverage counts lines of code run, and branch coverage counts the decision outcomes (such as both sides of an if statement) taken.
Database Migration	A versioned script that changes the database schema in a controlled, repeatable way.
Docker	A tool that packages an application and its dependencies into containers so that it runs the same way on any server.
Flask	The Python web framework used to build the backend.
Frontend	The browser-based user interface, built with React.
Hash (Salted Hash)	A one-way transformation of a password into a fixed string, combined with a random value (the salt) so that identical passwords produce different results. Passwords are stored only in this form.
MySQL	The relational database used by the system in development, testing and deployment.
ORM	Object-Relational Mapper: a library that lets the backend work with database tables as program objects.
Protected Request	A request for a function or data that requires the user to be authenticated and authorised.
React	The JavaScript library used to build the frontend.
REST API	An API in which the frontend requests data and actions from the backend over HTTP using standard methods.

Term	Definition
Self-hosted Deployment	Running the system on servers owned and operated by the institution. This is the deployment model chosen for the system (Appendix D).
Serverless	A cloud model in which code runs as short-lived functions without a continuously running server. Considered and not chosen (Appendix D).
Session Identifier	An opaque random value given to the browser to refer to a user's authenticated session. It contains no role or personal data.
Session Store	The backend's server-side storage of authenticated session state.
Static Code Analysis	Automated inspection of source code for defects and vulnerabilities, run in the CI pipeline.

A.7 Project and Document Terms

Term	Definition
Business Rule	An operating principle of the system, such as which role may do what and under which circumstances. Business rules are listed in Section 6.5 and numbered BR-nnn.
Hypothetical Stakeholder	The fictional institutional representative used in place of a real customer to ground scope and design decisions. See Appendix D.
Requirement ID	The unique tag given to each functional requirement, in the form REQ-XXX-nnn. The prefixes are ONB (onboarding), REC (edit records), ATT (attendance), GRD (grades), RET (retrieval), RBAC (role-based access), SEC (security, Section 6.3), SAF (safety, Section 6.2), QUA (quality attributes, Section 6.4) and OTH (other requirements, Section 7). Business rules use the form BR-nnn.
Stimulus/Response Sequence (SR)	A numbered pairing of a user action (stimulus) and the system's behaviour in response, used to describe each feature.
TBD	To be determined: information not yet available at the time of writing.

A.8 Abbreviations

Abbreviation	Expansion
CI/CD	Continuous Integration / Continuous Deployment
DPDP Act	Digital Personal Data Protection Act, 2023 (India)
HTTP(S)	Hypertext Transfer Protocol (Secure)
IST	Indian Standard Time
JSON	JavaScript Object Notation
NFR	Nonfunctional Requirement
SQL	Structured Query Language
SRS	Software Requirements Specification
UI	User Interface
UTC	Coordinated Universal Time
WCAG	Web Content Accessibility Guidelines

Appendix B: Field Layouts

Student

Field	Length	Data Type	Description	Is Mandatory

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID

Appendix D: Stakeholder Discussion Log

This project had no real customer, so a hypothetical stakeholder was used to ground scope and design decisions in the same back-and-forth a real software engineering team would have. The discussions below were conducted over roughly four weeks, and the decisions recorded here are referenced from Sections 1.3, 2.4, 2.5 and 2.6 of this SRS.

Persona

Name / Role: Dr. Kobe Bryant, Vice Chancellor

Context: Dr. Bryant oversees a university with approximately 10,000 students and staff combined. Student records, attendance and grades are currently maintained across spreadsheets and departmental registers, which makes consolidated reporting slow and access control inconsistent. He has commissioned a single Student Record System for the whole institution.

Discussion 1: Discovery Call (Monday, 14 September 2026)

Topics covered: current pain points, user roles, must-have features.

Q: What problems with the current record-keeping process are driving this request?

A: Records are scattered across departments. Producing a consolidated report for a single student means collecting data from several offices, and there is no reliable way to tell who changed a record or when. Attendance is often entered days late.

Q: Who will use the system, and what should each group be able to do?

A: Three groups: administrative staff who register students and maintain records, faculty who mark attendance and grades for their own courses, and students who should be able to view only their own records. Faculty must not be able to see or edit other faculty's courses.

Q: Are there rules on attendance entry that the system should enforce?

A: Yes. Attendance should only be markable for a class session that has actually been scheduled, a student cannot be marked present in two overlapping sessions, and faculty should be able to correct an entry only within a limited window after the session. After that, only an administrator can amend it, with the change logged.

Q: How are grades assigned today?

A: Each course is evaluated against a rubric of weighted components such as assignments, internal tests and a final exam. Rubrics differ between courses, so the system cannot assume a single fixed scheme.

Q: Who should be allowed to define or change a course's rubric?

A: Most courses are taught by more than one faculty member, and the rubric has to be the same for every student in the course. For each course, one of the faculty teaching it will be designated as the anchor faculty, assigned by an administrator. Only the anchor faculty may create or alter that course's rubric. Other faculty teaching the course can view the rubric and enter grades against it, but cannot change it. Administrators can reassign the anchor faculty, for example if the person leaves or stops teaching the course, but they do not edit rubrics themselves.

Decisions made:

- Three user roles: Administrator, Faculty and Student, with role-based access enforced by the system.
- Core features confirmed: onboarding, editing records, attendance with constraints, rubric-based grading, retrieval, and role-based access.
- Attendance edit window and overlap rules will be specified as concrete requirements (see Section 5.3).
- The grading rubric will not be a fixed scheme. It is defined per course, and only the course's anchor faculty (designated by an Administrator from the faculty teaching the course) can create or alter it (see Section 5.4).
- Every change to a record, attendance entry or grade must be traceable to a user and a timestamp.

Discussion 2: Deployment and Constraints (Thursday, 24 September 2026)

Topics covered: cloud versus self-hosted deployment, cost, expected load, data sensitivity, technology stack.

Q: Should the system be hosted on a cloud provider such as AWS, GCP, Azure or Firebase, or run on the institution's own infrastructure?

A: We initially leaned towards the cloud because it avoids buying hardware and gives a public URL immediately. The team walked through the trade-offs, and several points changed our view.

First, cost. A cloud deployment carries a recurring bill for compute, managed database, storage and backups that grows with usage and never ends. The institution already owns server capacity and a data centre room with spare capacity, so self-hosting is a one-time setup effort with no ongoing subscription.

Second, maintenance. The institution's existing IT team already runs servers and databases for other systems, so operating one more containerised application is well within their abilities and does not require hiring or training.

Third, the main advantage of the cloud does not apply here. Cloud platforms are most valuable for load balancing and low latency across large geographic areas, but nearly all users of this system are students and staff who regularly visit the campus and can reach a server on the campus network. There is no geographically distributed user base to serve.

Q: If cloud had been chosen, would a conventional backend or a serverless framework have been preferred?

A: Serverless was discussed and set aside even before the cost argument. The system depends on session-based authentication, role checks on every request and validation of attendance rules, all of which are simpler to build, test and reason about in a conventional long-running backend. Cold starts would also work against the response-time target below.

Q: What scale should the system be designed for?

A: About 10,000 users in total. Most of the time only a small fraction will be active, but there are predictable peaks: faculty marking attendance at the start and end of class periods, and students checking results when grades are published. The system should handle a few hundred concurrent users comfortably during these peaks.

Q: What are the expectations on response time?

A: Routine actions such as marking attendance or looking up a student should feel instant. A target of under two seconds for record retrieval under normal load was agreed.

Q: Are there data privacy or retention constraints?

A: Student records are personal data and must be protected accordingly, including compliance with India's Digital Personal Data Protection Act, 2023. Access must be limited by role, passwords must never be stored in plain text, and the database must be backed up regularly. Records are to be retained for the duration of a student's enrolment and for a period afterwards, as set by the university's records policy.

Q: Which technology stack should be used?

A: The institution has no preference, provided it is open source and maintainable by the existing IT team. The team proposed React for the frontend, Flask for the backend and a relational database, since student, course, attendance and grade data is inherently relational. MySQL will be used as the database in every environment (development, testing and deployment), so that the system behaves the same everywhere and the IT team only has to maintain one database technology.

Decisions made:

- Deployment model: self-hosted on the institution's own infrastructure, packaged with Docker so the IT team can deploy and update it consistently. Cloud and serverless options were considered and rejected for cost, operational simplicity and lack of a geographically distributed user base.
- Technology stack: React (frontend), Flask (backend), MySQL (database, used in development, testing and deployment).
- Design capacity: approximately 10,000 total users, with a few hundred concurrent users at peak.
- Performance target: record retrieval in under 2 seconds under normal load.
- Security: hashed passwords, role-based access enforced on the server, audit trail of changes, regular database backups.

Discussion 3: Sign-off (Tuesday, 6 October 2026)

Topics covered: final scope confirmation, items excluded from this release, acceptance.

Q: Having reviewed the scope, is there anything that must be added before work begins?

A: No additions. The six features agreed in Discussion 1 cover the immediate need.

Q: Is there anything about how grades are published or corrected that should be settled before work begins?

A: Grades should stay hidden from students until they are final, so there must be a single point at which they are released. The anchor faculty will publish a course's grades. After publication, only the anchor faculty may correct a grade, and a written reason must be recorded so that every correction can be explained later. The rubric itself should be locked once grades are published.

Q: Are there features that can safely be deferred?

A: Yes. Fee management, timetable generation, a parent portal and integration with the university's single sign-on service are out of scope for this release. They can be considered in a later phase, and the design should not make them hard to add.

Q: Is the proposed deployment plan acceptable to the institution's IT team?

A: Yes. The IT team has reviewed the self-hosted, containerised approach and confirmed they can host and maintain it on existing infrastructure.

Decisions made:

- Scope for this release is the six confirmed features: onboarding, editing records, attendance with constraints, rubric-based grading, retrieval, and role-based access.
- Grades are visible to students only after the anchor faculty publishes them. After publication, only the anchor faculty may amend a grade, with a recorded reason, and the rubric is locked.
- Out of scope: fee management, timetable generation, parent portal, single sign-on integration.
- The extensibility of the design to accommodate deferred features later is a stated design goal.
- The stakeholder approved the SRS scope, deployment model and technology stack as recorded in this appendix.